

Universidad ORT Uruguay
Facultad de Ingeniería

Diseño de Aplicaciones II
Obligatorio 1

Evidencia de Diseño y Especificación de la Api

Santiago Duró - 256325 - M6C

Lucas Facello - 298679 - M6C

Federico Katz - 257073 - M6B

Docentes M6C: Nicolás Fierro, Alexander Wieler

Docentes M6B: Daniel Acevedo, Federico González

Repositorio:

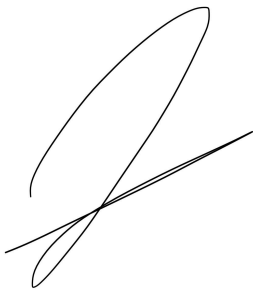
https://github.com/IngSoft-DA2/257073_256325_298679

8 de octubre, 2024

DECLARACIÓN DE AUTORÍA

Nosotros, Santiago, Federico, y Lucas, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el curso de Diseño de Aplicaciones II;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Santiago Duró

8 de octubre 2024



Lucas Facello

8 de octubre 2024



Federico Katz

8 de octubre 2024

Índice

Criterios Rest.....	4
Descripción del mecanismo de autenticación de requests.....	5
LogIn (Autenticación).....	5
LogOut (Desautenticacion).....	6
Descripción general de códigos de error.....	6
Descripción de los resources de la API.....	7
Administradores.....	7
Autenticaciones.....	8
Cámaras.....	8
Desautenticaciones.....	8
Dispositivos.....	9
Dispositivos Soportados.....	9
Dispositivos Hogar.....	10
Dueños Empresa.....	10
Dueños Hogar.....	11
Empresas.....	11
Hogares.....	12
Notificaciones.....	13
Sensores.....	14
Usuarios.....	14

Criterios Rest

A lo largo del desarrollo del proyecto analizamos varios factores para poder cumplir con los requerimientos que debe tener una API REST. A continuación describimos algunos de estos aspectos:

- Un principio esencial de REST dice que cada solicitud debe ser independiente y contener toda la información necesaria para completarse. Al no conservar un estado activo del lado del servidor, el cliente deberá enviar toda la información requerida y cada petición al servidor será independiente.
- En principio, nos aseguramos que la parte del servidor cumpliera la arquitectura cliente-servidor, lo que permite escalar la plataforma sin depender de la interfaz de usuario.
- Todos los recursos expuestos fueron nombrados con sustantivos en plural, se tomó esta decisión y se fue consistente durante el desarrollo de todos los endpoints del sistema.
- Las entidades de nuestro sistema se modelan como recursos, es decir que cada recurso tiene un controlador específico que lo representa. Con esto, buscamos seguir el principio de exposición de recursos (URL claras y consistentes).
- Para representar las acciones de cada uno de los endpoints implementados, utilizamos los verbos HTTP, para mantenernos dentro de los criterios REST.
- Para el manejo de errores se utilizaron respuestas con su respectivo código de error HTTP como también una descripción de la situación dada.
- El uso de rutas estandarizadas y endpoints predecibles para gestionar los diferentes recursos implementados sigue el principio de una interfaz uniforme. De esta manera, buscamos que los clientes puedan entender y manipular los recursos sin tener que conocer detalles internos de implementación.
- Uso de los filtros de autenticación y autorización, que nos permiten controlar el acceso a los recursos de forma centralizada. De esta manera nos podemos asegurar que solo los usuarios con los permisos adecuados pueden realizar acciones sobre determinados recursos.

Descripción del mecanismo de autenticación de requests

En el desarrollo de nuestra API, nos enfocamos en garantizar la seguridad del sistema a través de un proceso de autenticación. Decidimos implementar esto debido a las funcionalidades especificadas y los requerimientos pedidos, ya que algunas de estas implican un acceso restringido.

La autenticación se implementó utilizando un esquema basado en tokens, el cual permite identificar de manera segura a los usuarios que interactúan con la API, manteniendo la integridad y confidencialidad de la información. A continuación, se describen los detalles y las decisiones tomadas para la implementación de este sistema de autenticación.

LogIn (Autenticación)



El cliente envía una solicitud POST al endpoint /autenticaciones con las credenciales del usuario (correo electrónico y contraseña). El usuario que va a iniciar sesión ya debe estar previamente creado por el administrador. Luego, el controlador Autenticacion maneja esta solicitud.

El body de la request es el siguiente:

```
1  {  
2    "Email": "coco@gmail.com",  
3    "Contraseña" : "coco1234."  
4  }
```

Dentro del método Autenticar, se recibe una instancia de CrearSolicitudAutenticacion, que contiene los campos Email y Contraseña. Este objeto es procesado y validado, asegurándose de lo siguiente:

- Se valida que tanto el correo como la contraseña no estén vacíos.
- Se verifica que las credenciales proporcionadas coincidan con un usuario registrado en el sistema.
- Se comprueba que el usuario no esté ya en sesión para evitar múltiples inicios de sesión simultáneos.

Si alguna de estas validaciones falla, se lanzan excepciones adecuadas para notificar al cliente de errores en la solicitud enviada.

Si las validaciones son exitosas, se recupera el usuario con un método donde obtenemos el usuario a través de su email y se agrega una nueva sesión para el usuario. Al agregar una sesión, la misma devuelve un token generado, el cual se utiliza para identificar y autorizar al usuario en las futuras solicitudes.

Por último, el controlador devuelve un objeto `CrearRespuestaAutenticacion`, que contiene el token generado para el usuario autenticado.

LogOut (Desautenticacion)

Implementamos un mecanismo para las desautenticaciones, el cual permite a los usuarios cerrar su sesión de manera segura. Cuando un usuario ya no necesite interactuar con el sistema, su sesión se cierra correctamente y su token deja de estar vigente, evitando así posibles accesos no autorizados en el futuro.

Para cumplir con este requerimiento, hemos decidido implementar un sistema en el que el usuario, al realizar una solicitud de desautenticación, valida su identidad a través de un email, el cual se verifica en la lógica de negocio. Posteriormente, se cierra la sesión del usuario si está activa. Este enfoque asegura que el sistema mantenga su integridad y el control adecuado de las sesiones de los usuarios.

En la implementación, la solicitud de desautenticación (representada por la clase `CrearSolicitudDesautenticacion`) valida los datos ingresados, como el correo electrónico del usuario, antes de proceder a cerrar la sesión si es que el usuario está actualmente autenticado.

Descripción general de códigos de error

El protocolo HTTP utiliza códigos de respuesta numéricos, los cuales brindan información del resultado de la solicitud realizada a un determinado servidor web. Para el desarrollo de nuestra Api, decidimos utilizar códigos de la familia de los 2xx, 4xx y 5xx.

- **200 OK:** Este código lo utilizamos para comunicar que la solicitud ha sido exitosa y el servidor ha enviado correctamente el contenido solicitado.
- **400 Bad Request:** Lo utilizamos cuando el servidor no puede procesar la solicitud debido a un error del cliente, como datos mal formateados o parámetros inválidos.

- **401 Unauthorized:** Utilizamos este código de error para indicar que la solicitud hecha por el cliente carece de credenciales de autenticación válidas.
- **403 Forbidden:** Aunque el servidor entiende la solicitud y el cliente está autenticado, el acceso al recurso está prohibido. Este código de error lo utilizamos cuando el usuario no tiene los permisos adecuados para acceder al recurso solicitado.
- **404 Not Found:** Este código es utilizado por Postman, y se utiliza cuando el servidor no puede encontrar el recurso solicitado. Este error ocurre cuando el cliente intenta acceder a una URL que no existe o el servidor no logró encontrar el objeto en la base de datos.
- **500 Internal Server Error:** Indica que algo salió mal en el servidor mientras intentaba procesar la solicitud.

Una de las posibles correcciones a futuro es la utilización del código 201: Created.

Descripción de los resources de la API

Desarrollamos nuestra Api con diferentes resources.

El URL base utilizado es: <https://localhost:7265/>

Administradores

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un administrador.	POST	/administradores	Token de administrador.	Nombre: <i>string</i> Apellido: <i>string</i> Email: <i>string</i> Contraseña: <i>string</i>	200 400 401 403 500
Eliminar un administrador.	DELETE	/administradores /{email}	Token de administrador.		200 400 401 403 500

Autenticaciones

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Utilizado para obtener un token de la sesión de un usuario.	POST	/autenticaciones		Email: <i>string</i> Contraseña: <i>string</i>	200, Token : <i>string</i> 400 500

Cámaras

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear una cámara.	POST	/camaras	Token de dueño de empresa.	Nombre: <i>string</i> NumeroModelo: <i>int</i> Descripcion: <i>string</i> fotografias: <i>List<Fotografias></i> <i>[{</i> <i>Url :</i> <i>string</i> <i>EsPrincipal:</i> <i>bool</i> <i>}]</i> DetectaMovimiento: <i>bool</i> DetectaPersona: <i>bool</i> UsoInterior: <i>bool</i> UsoExterior: <i>bool</i>	200 400 401 403 500

Desautenticaciones

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Utilizado para eliminar la sesión de un usuario.	POST	/desautenticaciones	Token	Email: <i>string</i>	200 400 401 500

Dispositivos

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Obtener un listado de todos los dispositivos registrados.	GET	/dispositivos	NumeroDe Pagina: int (si <= 0 se le asigna 1) TamañoDe Pagina: int (si <= 0 se le asigna 10) Nombre Dispositivo: string Numero Modelo: int Nombre Empresa: string Tipo Dispositivo: string	Token		200: [{ Nombre: string NumeroModelo: int FotoPrincipal: string NombreEmpresa: string }] 401 500

Dispositivos Soportados

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Obtener un listado de todos los tipos de dispositivos inteligentes que soportan el sistema.	GET	/dispositivos Soportados	Token		200: [{ Tipo: string }] 401 500

Dispositivos Hogar

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Un sensor de un hogar se abre y genera notificación.	POST	/dispositivosHogar/{id}/abrir			200 400 500
Un sensor de un hogar se cierra y genera notificación.	POST	/dispositivosHogar/{id}/cerrar			200 400 500
Un dispositivo de un hogar se pone en línea.	POST	/dispositivosHogar/{id}/conectar	token		200 400 401 500
Un dispositivo de un hogar deja de estar en línea.	POST	/dispositivosHogar/{id}/desconectar	token		200 400 401 500
Una cámara de un hogar detecta movimiento y genera notificación.	POST	/dispositivosHogar/{id}/movimiento			200 400 500
Una cámara de un hogar detecta a una persona y genera notificación.	POST	/dispositivosHogar/{id}/persona			200 400 500

Dueños Empresa

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un dueño de empresa.	POST	/dueños Empresa	Token de administrador	Nombre: <i>string</i> Apellido: <i>string</i> Email: <i>string</i> Contraseña: <i>string</i>	200 400 401 403 500

Dueños Hogar

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un dueño de hogar.	POST	/dueñosHogar		Nombre: <i>string</i> Apellido: <i>string</i> Email: <i>string</i> Contraseña: <i>string</i> FotoPerfil: <i>string</i>	200 400 500

Empresas

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Crear empresa	POST	/empresas		Token de dueño empresa.	Nombre: <i>string</i> Logotipo: <i>string</i> Rut: <i>string</i>	200 400 401 403 500
Listar empresas	GET	/empresas	NumeroDe Pagina: <i>int</i> (si <= 0 se le asigna 1) TamañoDe Pagina: <i>int</i> (si <= 0 se le asigna 10) Nombre Empresa: <i>string</i> Nombre Completo Dueño: <i>string</i>	Token de administrador		200 [Nombre: <i>string</i> Logotipo: <i>string</i> Rut: <i>string</i> NombreDueño: <i>string</i>] 401 403 500

Hogares

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un hogar	POST	/hogares	Token de dueño hogar	Calle: <i>string</i> NumeroPuerta: <i>int</i> Latitud: <i>int</i> Longitud: <i>int</i> CantMiembros Soportados: <i>int</i>	200 400 401 403 500
Agregar miembro a un hogar	POST	/hogares/{id} /miembros	Token de dueño hogar	Email: <i>string</i> PermisoAsociar Dispositivos: <i>bool</i> PermisoListar Dispositivos: <i>bool</i> Permiso Notificaciones: <i>bool</i>	200 400 401 403 500
Asociar un dispositivo a un hogar.	POST	/hogares/{id} /dispositivos	Token de dueño hogar	DispositivoId: <i>string</i>	200 400 401 403 500
Listar todos los dispositivos asociados a un hogar.	GET	/hogares/{id} /dispositivos	Token de dueño hogar		200: [{ Nombre: <i>string</i> NumeroModelo: <i>int</i> FotoPrincipal: <i>string</i> Nombre Empresa: <i>string</i> EstaConectado: <i>bool</i> }] 400 401

					403 500
Listar todos los miembros asociados a un hogar.	GET	/hogares/{id}/miembros	Token de dueño hogar		200 [NombreCompleto: string FotoPerfil: string PermisoAsociar Dispositivos: bool PermisoListar Dispositivos: bool Permiso Notificaciones: bool] 400 401 403 500

Notificaciones

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Listar notificaciones	GET	/notificaciones	Tipo Dispositivo: string FechaDe Creacion: DateTime (formato mm/dd/aaaa) Leida: bool	Token		200 [DispositivoHogarId: string DispositivoId: string Evento: string FueLeida: bool FechaHora: string] 401 500

Sensores

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un sensor.	POST	/sensores	Token de dueño empresa.	Nombre: <i>string</i> NumeroModelo: <i>int</i> Descripcion: <i>string</i> fotografias: <i>List<Fotografias></i> [Url: <i>string</i> EsPrincipal: <i>bool</i>]]	200 400 401 403 500

Usuarios

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Obtener un listado con todos los usuarios registrados.	GET	/usuarios	NumeroDePagina: <i>int</i> (si <= 0 se le asigna 1) TamañoDePagina: <i>int</i> (si <= 0 se le asigna 10) Rol: <i>string</i> NombreCompleto: <i>string</i>	Token de administrador.		200 [Nombre: <i>string</i> Apellido: <i>string</i> NombreCompleto: <i>string</i> TipoRol: <i>string</i> FechaCreacion: <i>DateTime</i>] 401 403 500

El Token que se incluye en el Header al realizar estos endpoints debe pertenecer a una sesión autenticada, en la cual el usuario logueado tenga una sesión activa en la lista de sesiones. Al desautenticarse, el token deja de ser válido, por lo que el usuario deberá autenticarse nuevamente para ejecutar acciones que requieran header. La lista de sesiones comienza vacía cada vez que se corre la aplicación, por lo que es necesario autenticarse en cada ejecución.